Lista NP-completo
Analise de Algoritmos
Felipe Echeverria Vilhalva

1. Informalmente, defina as classes P, NP e NP-Completo de problemas.

- **Classe P**: Consiste nos problemas de decisão que podem ser solucionados (resolvidos) em **tempo polinomial**. Informalmente, são problemas tratáveis ou "fáceis".
- **Classe NP**(Não-determinísticos Polinomais): Consiste nos problemas de decisão que são verificaveis em **tempo polinomial**. Se um "certificado" (solução candidata) for fornecida, pode-se verificar se ele está correto em tempo polinomial no tamanho da entrada.
- **Classe NP-Completo**(NPC): informalmente, um problema está na classe NPC se ele estiver em NP e for tão "dificil" quanto qualquer problema em NP. São os problemas mais dificeis na classe NP.

2. O que é apresentado quando se prova que um problema é NP-Completo ? caiu 2024

Quando se prova que um problema é NP-completo,está-se fazendo uma afirmação sobre o quão dificil ele é (ou o quão dificil se pensa que é). Apresenta-se que ***provavelmente não existirá um algoritmo eficiente*** (de tempo polinomial) para resolve-lo.

3. O que é um problema de Otimização ?

Um problema de otimização é aquele em que, para cada solução viável ("legal"), há um valor associado, e o objetivo é encontrar uma solução viável com o **melhor valor** (máximo ou minimo).

4. O que é um problema de Decisão ?

Um problema de decisão é aquele em que a resposta é simplesmente "sim" ou "não" (ou, mais formalmente, "1" ou "0"). A teoria da NP-completude restringe a atenção a esses problemas.

5. Identifique a relação existente entre Problemas de Otimização e Problemas de Decisão que pode ajudar na prova de um problema NP-Completo. caiu 2024

A relação é que, normalmente, um problema de otimização pode ser transformado em um **problema de decisão relacionado** ao impor um **limite(*k*)** ao valor a ser otimizado.

ssa relação é vantajosa porque o problema de decisão é, em certo sentido, **"não mais difícil"** que o problema de otimização. Isso significa que, **se o problema de otimização é fácil, seu problema de decisão relacionado também é fácil**. Ao provar que o problema de decisão é difícil (NP-Difícil), implicitamente se estabelece que o problema de otimização também é difícil.

6. O que é um algoritmo de redução? Como ele nos ajuda na prova de um problema NP Completo? cai

- **Algoritmo de Redução:** É um algoritmo de **tempo polinomial** (F) que calcula uma **função de redução** (f). Este algoritmo transforma qualquer instância α de um problema A em uma instância β de um problema B.
- **Ajuda na Prova de NPC:** A redução em tempo polinomial ($L' \leq PL$) permite provar formalmente que um problema (L') **não é mais difícil de resolver** que outro (L), a não ser por um fator polinomial. Ao reduzir um problema NP-Completo conhecido (L') para o problema que está sendo analisado (L), provamos implicitamente que L é **NP-Difícil** (tão difícil quanto qualquer problema em NP).

7. O que é oproblema de satisfação de circuito ?

O problema da satisfação do circuito (CIRCUIT-SAT) pergunta se, dado um **circuito combinacional lógico** composto de portas AND, OR e NOT, existe algum conjunto de **entradas booleanas** que faz com que sua saída seja **1 (verdadeiro)**.

8. O que é um Problema Abstrato?

Um problema abstrato Q é definido como uma **relação binária** em um conjunto I de instâncias de problemas e um conjunto S de soluções de problemas.

9. O que é um Problema de Decisão Abstrato?

Um problema de decisão abstrato é um problema abstrato que pode ser visto como uma **função** que mapeia o conjunto de instâncias I para o conjunto de soluções **{0,1}** (sim/não).

10. O que é um Problema Concreto ?

Um problema concreto é aquele cujo conjunto de instâncias é um conjunto de **strings binárias**. Para que algoritmos de computador possam solucionar problemas abstratos, suas instâncias precisam ser representadas (codificadas) em strings binárias.

11. Defina formalmente a Classe P de Problemas.

A **Classe P** de problemas é o conjunto de problemas de decisão concretos que são **solucionáveis** (decidíveis) em **tempo polinomial**.

$$P=\{L \subset \{0,1\}^* : \text{existe um algoritmo } A \text{ que decide } L \text{ em tempo polinomial}\}$$

12. O que é algoritmo de verificação ?

Um algoritmo de verificação é um algoritmo de **dois argumentos** A: uma string de entrada comum x e uma string binária y chamada de **certificado**. O algoritmo A verifica se, para a string de entrada x, existe um certificado y tal que $A(x,y)=1$. true

13. Dizemos que intuitivamente um algoritmo A verifica uma linguagem L se ?

cai

Intuitivamente, um algoritmo A verifica uma linguagem L se:

- Para qualquer sequência $x \in L$, existe um **certificado** y que A pode usar para provar que $x \in L$.
- Para qualquer sequência $x \in! L$, **não deve haver certificado** que possa enganar o algoritmo A a acreditar que $x \in L$.

14. Defina formalmente a Classe NP de problemas.

A **Classe NP** de complexidade é a classe de linguagens que pode ser **verificada** por um algoritmo de **tempo polinomial**.

Formalmente, uma linguagem L pertence a NP se e somente se existir um algoritmo de tempo polinomial de duas entradas A e uma constante c tal que:

$L=\{x \in \{0,1\}* \mid \text{existe um certificado } y \text{ com } |y|=O(|x|^c) \text{ tal que } A(x,y)=1\}$

15. Intuitivamente dizemos que, um problema Q pode ser reduzido a outro problema Q' se?

cai

Intuitivamente, um problema Q pode ser reduzido a outro problema Q' se qualquer instância de Q puder ser **"facilmente reformulada"** como uma instância de Q', cuja solução fornece uma solução para a instância de Q.

16. Usando a estrutura de linguagens formais para problemas de decisão, quando podemos dizer que uma linguagem L1 é redutível em tempo polinomial a outra linguagem L2, L1 ≤p L2 ?

Dizemos que uma linguagem $L1$ é redutível em tempo polinomial a uma linguagem $L2$, escrita $L1 \leq pL2$, se existir uma **função f computável em tempo polinomial** tal que:

$$\text{Para todo } x \in \{0,1\}*, x \in L1 \text{ se e somente se } f(x) \in L2$$

17. O que é função de redução?

A **função f** que transforma uma instância de um problema em outra, satisfazendo a condição de equivalência de respostas ($x \in L1$ *se e somente se* $f(x) \in L2$), é chamada de **função de redução**.

18. O que é um algoritmo de redução ?

Um algoritmo de redução é um algoritmo de **tempo polinomial (F)** que **calcula a função de redução f**.

19. Defina formalmente a Classe de problemas NP-Completo.

Uma linguagem $L \subset \{0,1\}*$ é **NP-Completa** se satisfizer as seguintes duas condições:

- ***$L \in NP$.***
- ***$L' \leq pL$ para todo $L' \in NP$. (Ou seja, L é NP-Difícil).***

20. Liste os passos para provar que um problema pertence a classe de Problemas NP-Completo.

cai

Para provar que uma linguagem L é NP-Completa, usando um problema NP-Completo conhecido L' como base (Metodologia do Lema 1), os passos são:

1. Prove que $L \in NP$. (Mostre que L é verificável em tempo polinomial).
2. Selecione uma linguagem **NP-Completa conhecida (L')**.
3. Descreva um algoritmo que calcula uma função f mapeando cada instância x de L' para uma instância $f(x)$ de L. redução
4. Prove que a função f satisfaz: ***$x \in L'$ se e somente se $f(x) \in L$*** *para todo x*. (Equivalência de respostas).
5. Prove que o algoritmo que calcula f é executado em **tempo polinomial**. (Isso demonstra que $L' \leq pL$, ou seja, L é NP-Difícil).